COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	8/27/2026	**DATE SUBMITTED:**	8/27/2026
NAME:	Alvarez, Joshua Caleb P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 6.1 **Searching Techniques**

1. PROCEDURE OUTPUT

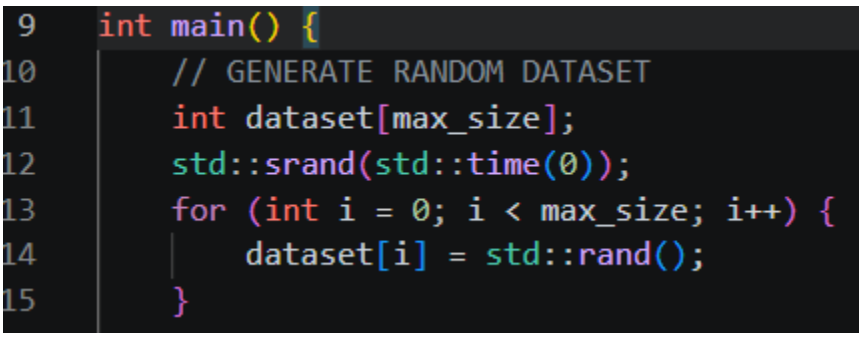
Screenshot	
Observation	Based on the screenshot, I made an array named 'dataset' that has stored 50 random numbers using max_size. For the random generation of numbers every time we execute our program, we have used srand(time(0)). Using a 'for' loop, we have made, stored, and displayed these numbers in the array.

Table 6-1. Data Generated and Observations.

Screenshot

```
5   // A.1 LINEAR SEARCH - ARRAY
6   void linearSearch(int arr[], int n, int item) {
7       int i = 0;
8       while (i < n) {
9
10      if (item == arr[i]) {
11          std::cout << "Searching is successful!" << std::endl;
12          std::cout << "Element found at index: "<< i << std::endl;
13              return;
14          }
15          i++;
16      }
17      std::cout << "Searching is unsuccessful!"<< std::endl;
18
```

Output

```
====================================
          GENERATED DATASET
====================================
2409 25949 6842 1768 9665 18044 13864 21630 18584 31079
4181 7530 22205 18049 11180 1977 5652 29678 11854 30552
6121 22170 16135 18228 7556 29821 11904 7583 11006 7198
16566 24566 15443 5462 10922 2432 20663 20311 6235 27034
16430 12268 20209 3345 3927 26627 16187 22224 13704 29032

====================================
        LINEAR SEARCH - ARRAY
====================================
Enter number to search: 2515
Searching is unsuccessful!
```

Observation

From my observation, the linear search is used to search for the number by testing each of the numbers, starting from the first element of the list, until it reaches the number I'm searching for. It is simple to code and understand, but it takes a long time if the number is towards the end of the list or is not in the list at all.

Table 6-2a. Linear Search for Arrays

Screenshot

```
20  // A.2 LINEAR SEARCH - LINKED LIST
21  template <typename T>
22  void linearLS(Node<T> *head, T dataFind) {
23      Node<T> *current = head;
24      while (current != NULL) {
25
26          if (current->data == dataFind) {
27              std::cout << "Searching is successful!"<< std::endl;
28              std::cout << "Element found: " << dataFind << std::endl;
29              return;
30          }
31          current = current->next;
32      }
33      std::cout << "Searching is unsuccessful!"
34                << std::endl;
```

```
36      // LINEAR SEARCH - LINKED LIST
37      Node<char> *name1 = new_node('J');
38      Node<char> *name2 = new_node('O');
39      Node<char> *name3 = new_node('S');
40      Node<char> *name4 = new_node('H');
41      Node<char> *name5 = new_node('U');
42      Node<char> *name6 = new_node('A');
43
44      name1->next = name2;
45      name2->next = name3;
46      name3->next = name4;
47      name4->next = name5;
48      name5->next = name6;
49      name6->next = NULL;
50
51      std::cout << "====================================="<< std::endl;
52      std::cout << "     LINEAR SEARCH - LINKED LIST"<< std::endl;
53      std::cout << "====================================="<< std::endl;
54      std::cout << "Name: ";
```

```
std::cout << std::endl;
// Successful search
std::cout << "Linked list Linear Search : ";
linearLS(name1, 'S');
// Unsuccessful search
std::cout << "Linked list Linear Search : ";
linearLS(name1, 'B');
std::cout << std::endl;
```

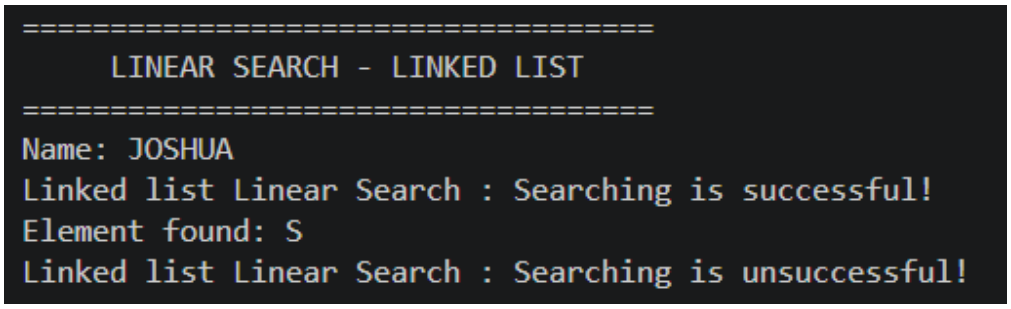
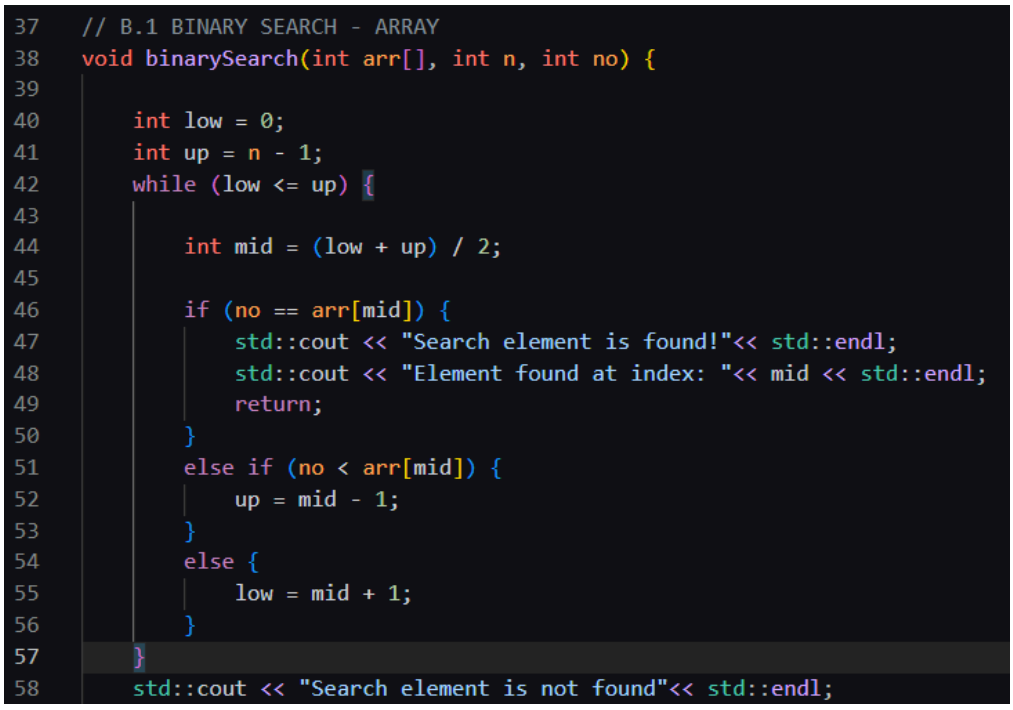
Output	
Observation	From my observation, Linear search behaves almost the same when applied to a linked list as well, as it involves going through each element one by one. In my program, I used a linked list that was generated using the letters of my first name, where each node was connected to another node. The search would begin from the first node and go on until the node containing the character being searched for is encountered.

Table 6-2b. Linear Search for Linked List

Screenshot	

```
37  // B.1 BINARY SEARCH - ARRAY
38  void binarySearch(int arr[], int n, int no) {
39
40      int low = 0;
41      int up = n - 1;
42      while (low <= up) {
43
44          int mid = (low + up) / 2;
45
46          if (no == arr[mid]) {
47              std::cout << "Search element is found!"<< std::endl;
48              std::cout << "Element found at index: "<< mid << std::endl;
49              return;
50          }
51          else if (no < arr[mid]) {
52              up = mid - 1;
53          }
54          else {
55              low = mid + 1;
56          }
57      }
58      std::cout << "Search element is not found"<< std::endl;
```

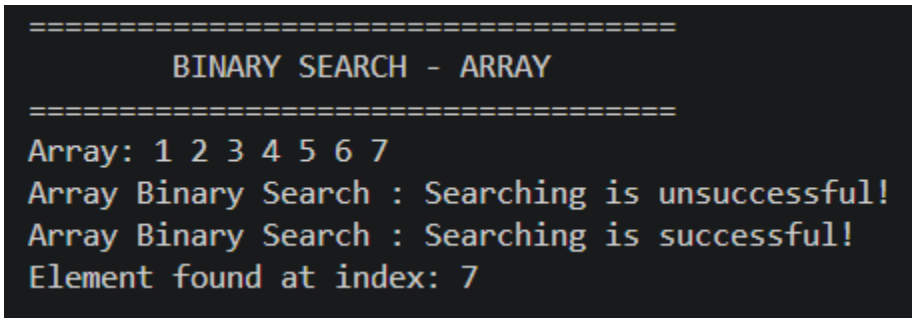
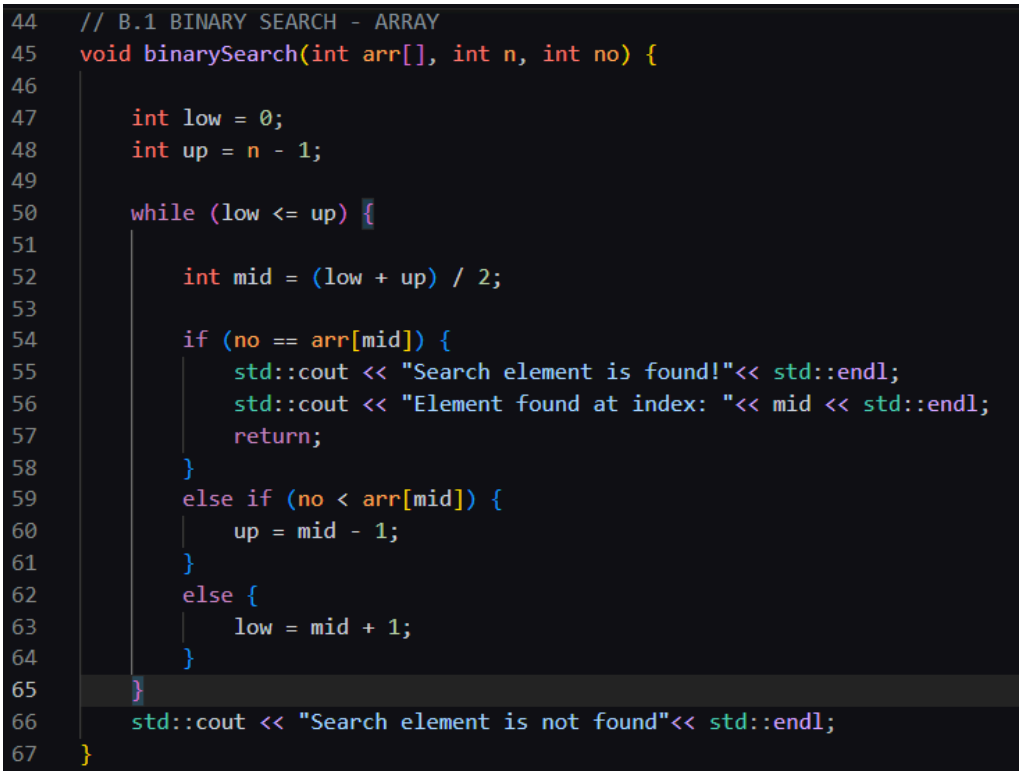
Output	
Observation	I observed that the binary search algorithm is such that it checks the middle element of a sorted array and decides on the half that needs to be searched further. I performed a search for 8 in my program. When this search process started, it checked 11 first, and since 8 was less than 11, it searched the lower half of the array. This was done until it located 8 at index 3. The reason binary search is faster than linear search is that it reduces the number of possible elements by half with each step.

Table 6-3a. Binary Search for Arrays

Screenshot	

```
44  // B.1 BINARY SEARCH - ARRAY
45  void binarySearch(int arr[], int n, int no) {
46
47      int low = 0;
48      int up = n - 1;
49
50      while (low <= up) {
51
52          int mid = (low + up) / 2;
53
54          if (no == arr[mid]) {
55              std::cout << "Search element is found!"<< std::endl;
56              std::cout << "Element found at index: "<< mid << std::endl;
57              return;
58          }
59          else if (no < arr[mid]) {
60              up = mid - 1;
61          }
62          else {
63              low = mid + 1;
64          }
65      }
66      std::cout << "Search element is not found"<< std::endl;
67  }
```

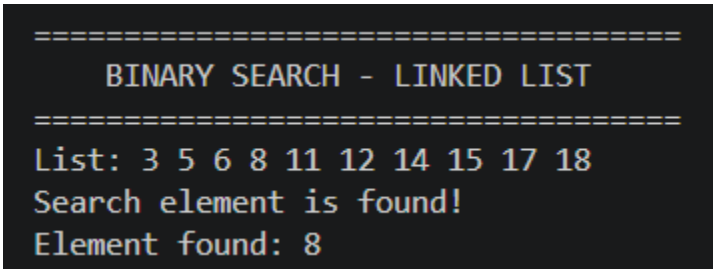
Output	
Observation	I have observed that binary search may also be carried out on a sorted linked list through finding the middle node of the list being searched. In my program, the slow and fast pointer technique is used to get the middle node. The value of the middle node is compared with the value being searched for, and the search proceeds on one side or another of the list. It has been noted that linked list binary search is more complex than array binary search.

Table 6-3b. Binary Search for Linked List

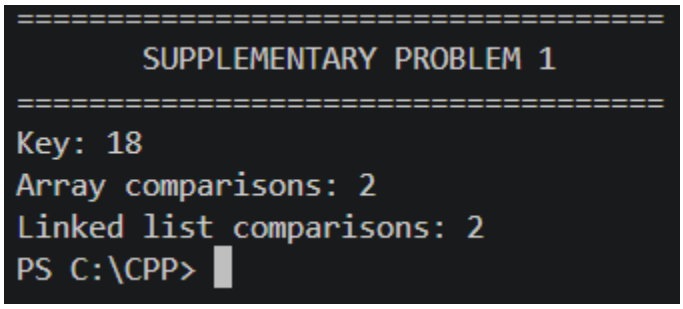
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Screenshot	

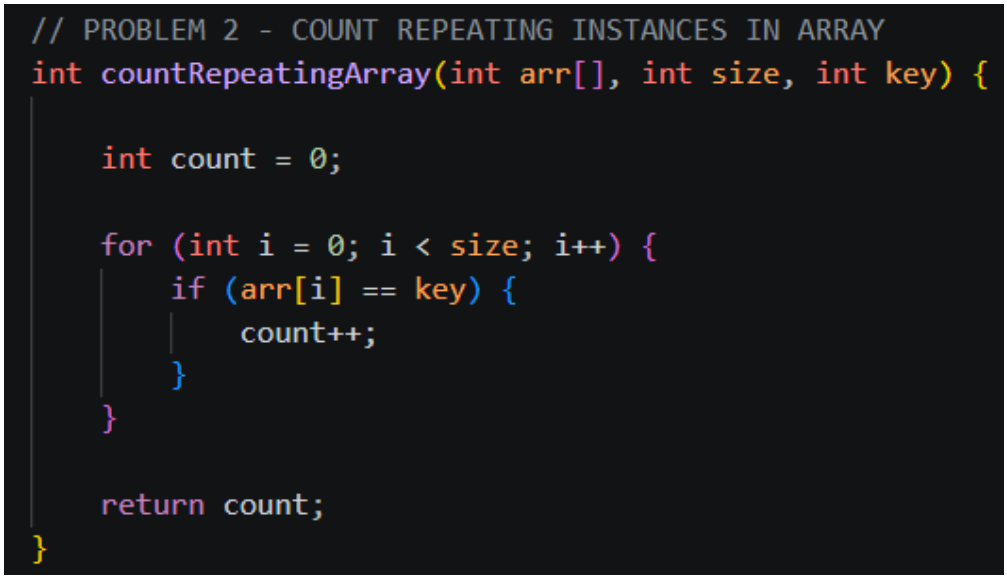
```
int main() {
    // PROBLEM 1
    // Given list
    int arr[10] = {
        15, 18, 2, 19, 18,0, 8, 14, 19, 14
    };
    int size = 10;
    int key = 18;

    // ARRAY SEARCH
    int arrayComparisons =
        linearSearchArray(arr, size, key);


    // CREATE LINKED LIST
    Node* head = new Node{15, NULL};
    head->next = new Node{18, NULL};
    head->next->next = new Node{2, NULL};
    head->next->next->next = new Node{19, NULL};
    head->next->next->next->next = new Node{18, NULL};
    head->next->next->next->next->next = new Node{0, NULL};
    head->next->next->next->next->next->next = new Node{8, NULL};
    head->next->next->next->next->next->next->next = new Node{14, NULL};
    head->next->next->next->next->next->next->next->next = new Node{19, NULL};
    head->next->next->next->next->next->next->next->next->next = new Node{14, NULL};
```

Output	
Observation	From my observations, binary search checks the middle element in the sorted array and then searches on either left or right depending on whether the middle element is smaller or larger than the element to be searched. For instance, I searched for 8 in my case. The initial middle element was 11, and therefore I searched on the left side since 8 is smaller than 11. I kept on searching using the middle elements until I found 8 at index 3.

PROBLEM 1

Screenshot	

```cpp
// PROBLEM 2 - COUNT REPEATING INSTANCES IN ARRAY
int countRepeatingArray(int arr[], int size, int key) {

    int count = 0;

    for (int i = 0; i < size; i++) {
        if (arr[i] == key) {
            count++;
        }
    }

    return count;
}
```

```cpp
    // PROBLEM 2
    int arrayCount =
        countRepeatingArray(arr, size, key);
    int linkedListCount =
        countRepeatingLinkedList(head, key);

    // DISPLAY RESULTS
    std::cout << std::endl;
    std::cout << "====================================" << std::endl;
    std::cout << "       SUPPLEMENTARY PROBLEM 2" << std::endl;
    std::cout << "====================================" << std::endl;
    std::cout << "Search element: " << key << std::endl;
    std::cout << "Repeating instances in array: "<< arrayCount << std::endl;
    std::cout << "Repeating instances in linked list: "<< linkedListCount << std::endl;
```

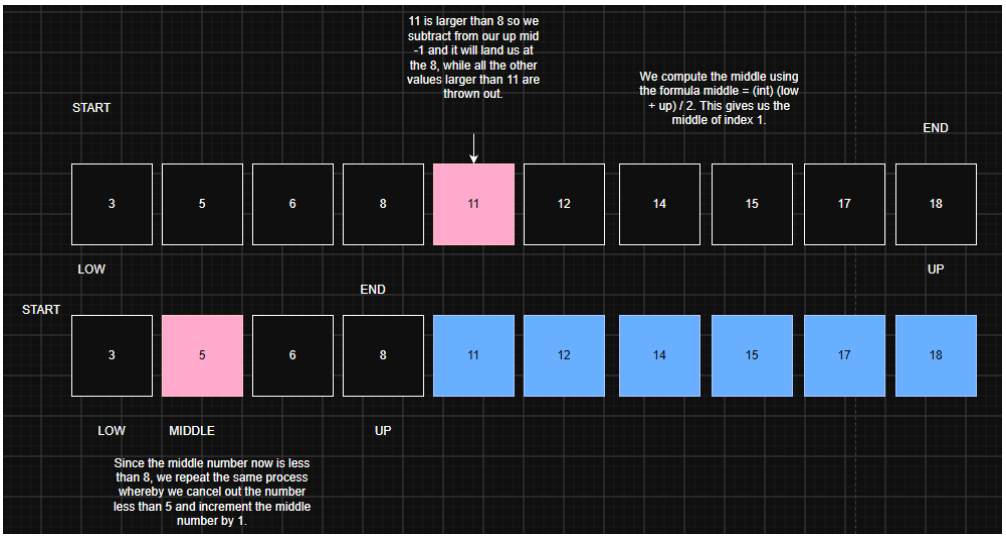
Output

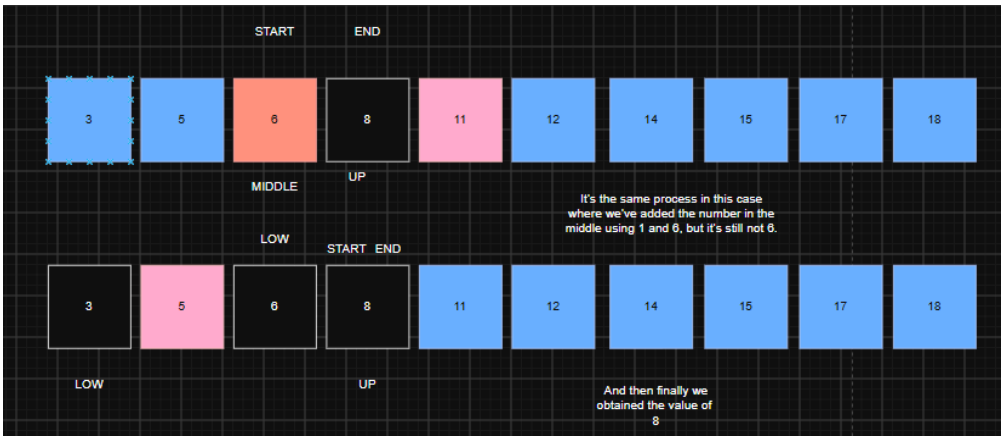
```
====================================
       SUPPLEMENTARY PROBLEM 2
====================================
Search element: 18
Repeating instances in array: 2
Repeating instances in linked list: 2
```

Observation

I observe that binary search can also be done on a sorted linked list through getting the middle node from the search area at any point. The middle node will be compared with the key, and then the process goes on to either the left part or the right part of the list. In my case, I used two pointers called the slow pointer and fast pointer to get the middle node.

Diagram


11 is larger than 8 so we subtract from our up mid -1 and it will land us at the 8, while all the other values larger than 11 are thrown out.
We compute the middle using the formula middle = (int) (low + up) / 2. This gives us the middle of index 1.
START
END
3 5 6 8 11 12 14 15 17 18
LOW
UP
END
START
3 5 6 8 11 12 14 15 17 18
LOW
MIDDLE
UP
Since the middle number now is less than 8, we repeat the same process whereby we cancel out the number less than 5 and increment the middle number by 1.


START
END
3 5 6 8 11 12 14 15 17 18
MIDDLE
UP
It's the same process in this case where we've added the number in the middle using 1 and 6, but it's still not 8.
LOW
START END
3 5 6 8 11 12 14 15 17 18
LOW
UP
And then finally we obtained the value of 8

Screenshot

```cpp
// PROBLEM 3 - BINARY SEARCH
int binarySearch(int arr[], int size, int key) {

    int low = 0;
    int up = size - 1;
    int comparisons = 0;
    while (low <= up) {
        int mid = (low + up) / 2;

        comparisons++;

        std::cout << "Iteration " << comparisons<< ": low = " << low<< ", mid = " << mid
                  << ", up = " << up<< ", value = " << arr[mid]<< std::endl;
        if (arr[mid] == key) {

            std::cout << "Search element is found!"<< std::endl;
            std::cout << "Element found at index: "<< mid<< std::endl;
            return mid;
        }
        else if (key < arr[mid]) {
            up = mid - 1;
        }
        else {
            low = mid + 1;
        }
    }
    std::cout << "Search element is not found"<< std::endl;
    return -1;
}
```

Output

```
====================================
       SUPPLEMENTARY PROBLEM 3
====================================
Sorted list: 3 5 6 8 11 12 14 15 17 18
Searching for: 8
Iteration 1: low = 0, mid = 4, up = 9, value = 11
Iteration 2: low = 0, mid = 1, up = 3, value = 5
Iteration 3: low = 2, mid = 2, up = 3, value = 6
Iteration 4: low = 3, mid = 3, up = 3, value = 8
Search element is found!
Element found at index: 3
```

Observation

The recursive binary search algorithm performs in the same manner as the conventional binary search algorithm; however, rather than having loops to perform the same operation again and again, the function recurses by reducing the searching range in every call. The function reduces the search range until it finds the element 8 at its place in the sorted list. Recursion has made the code shorter, but the conventional binary search process remains the same.

PROBLEM 3

Screenshot

```cpp
// PROBLEM 4 - RECURSIVE BINARY SEARCH
int recursiveBinarySearch(int arr[], int low, int up, int key) {

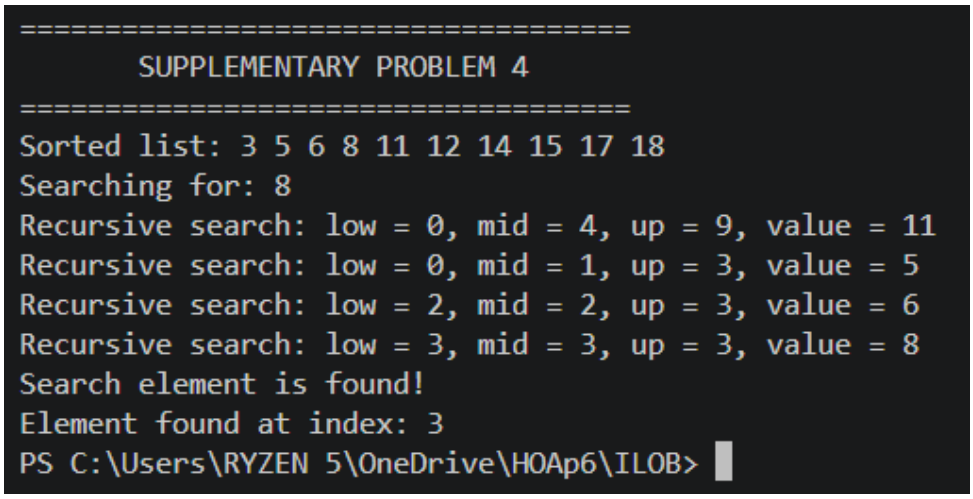
    if (low > up) {
        std::cout << "Search element is not found"<< std::endl;
        return -1;
    }

    int mid = (low + up) / 2;
    std::cout << "Recursive search: low = "<< low<< ", mid = "<< mid
              << ", up = "<< up<< ", value = "<< arr[mid]<< std::endl;
    if (arr[mid] == key) {
        std::cout << "Search element is found!"<< std::endl;
        std::cout << "Element found at index: "<< mid<< std::endl;
        return mid;
    }
    else if (key < arr[mid]) {
        return recursiveBinarySearch( arr,low,mid - 1,key
        );
    }
    else {
        return recursiveBinarySearch(arr,mid + 1,up,key);
    }
}
```

```cpp
    // PROBLEM 4

    std::cout << std::endl;
    std::cout << "====================================" << std::endl;
    std::cout << "       SUPPLEMENTARY PROBLEM 4" << std::endl;
    std::cout << "====================================" << std::endl;
    std::cout << "Sorted list: ";

    for (int i = 0; i < 10; i++) {
        std::cout << sortedArray[i] << " ";
    }
    std::cout << std::endl;
    std::cout << "Searching for: "<< binaryKey<< std::endl;
    recursiveBinarySearch(sortedArray,0,9,binaryKey);
    return 0;
}
```

Output	
Observation	From my observation, recursive binary search works in the same manner as normal binary search except for the fact that instead of looping through the search, the function is called recursively on the smaller search range. My search in the program was to find 8 in the sorted array. This search continues to divide the array into smaller sections until 8 is found at index 3. From my observation, the use of recursion causes the binary search procedure to be repeated.

PROBLEM 4

CONCLUSION:

In this activity, I learned how to use various types of searches in C++ through the implementation of arrays and linked lists. I learned how to perform a linear search as well as a binary search through the process of separating a sorted list of elements into two halves over and over again. I also learned how to keep the count of repeated elements and how to modify the binary search method to become recursive. As a result of the above activities, I have learned about the different approaches of performing searches on arrays and linked lists and how binary search is much faster when used on sorted data.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chatgpt for code assistance, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **It is used for grammar correction and code error assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Ai was used for code assistance, error correction, and grammar improvement while i completed and understand the required work myself

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.